PROPOSAL → PROOF

The QA framework, running on a real Sun.Studio game

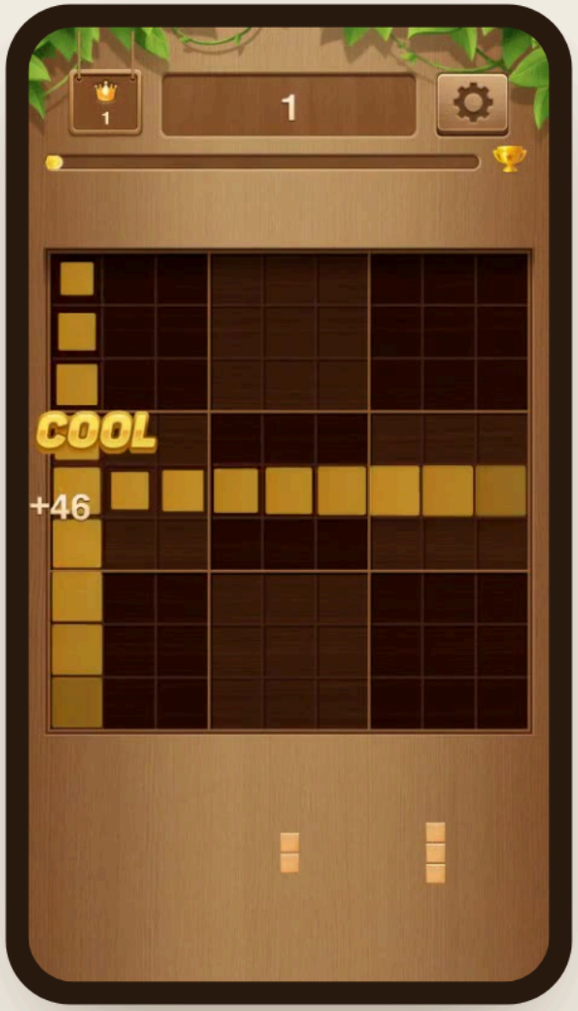
Wood Block Journey, HTML5 / Phaser 3. I had no source code and no spec, only the live build. The game went through analysis, test cases and three-tier automation.

32 / 32 automated checks green

47 test cases · 13 layers

Python · Pytest · Playwright · OpenCV

Thanh Nguyen (Leon) · QA Manager case study · Sun.Studio



WHY WOOD BLOCK JOURNEY

I picked the hard case on purpose

If the framework works here, it works on simpler games too.

Canvas

No DOM to hold on to

The whole game is drawn on one WebGL canvas, so no locator can reach a button. This is where most UI automation stops.

Black box

Live production build, minified

No source code, no test hooks, no spec. Everything had to be learned from the running game.

Real portal

Ads, SDK, analytics all live

Playhop / Yandex Games pre-rolls and interstitials appear at random. The same bundle also carries the FB Instant code path.

9×9

board · 3 pieces per hand · 50 piece variants

~1 day

from first look to a green suite

0

lines of game code needed

THE FLOW FROM THE PROPOSAL

One flow, four outputs

The same sequence I proposed for the 13-person team, run end to end on one game.

1

Test analysis

State model, observed rules, 13-layer risk matrix, questions for GD.

9 rules

5 open questions

2

Test cases

Notion schema: TC ID, layer, type, priority, suites, automation approach.

47 TCs

23 auto · 21 candidate · 3 charters

3

Automation

Three-tier ladder on one Python stack. A model of the rules is the oracle.

32 tests

bridge · scene · image

4

Report → Notion

JUnit + evidence per test, HTML report, Automation Run synced by TC ID.

9m 42s

full run, video per test

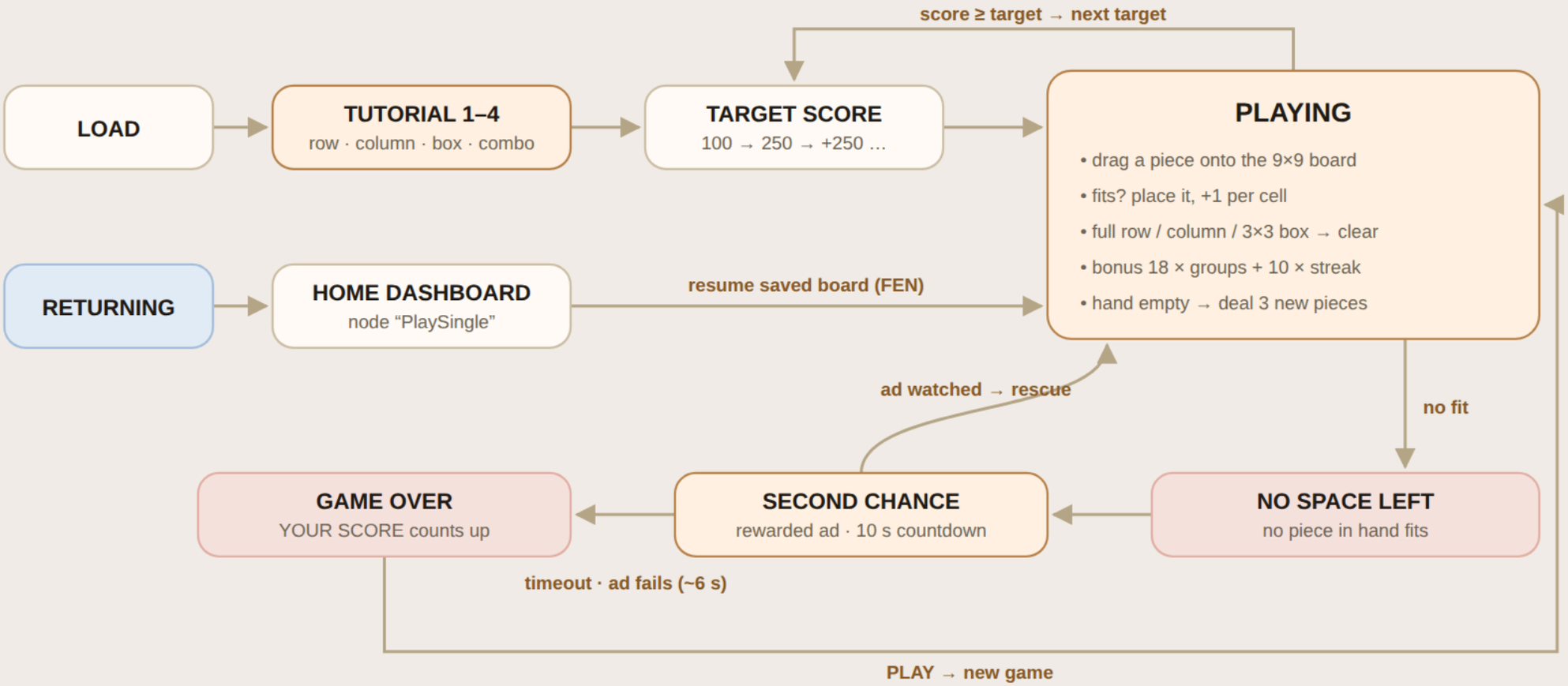
AI + human gate

Steps 1–2 follow the **game-test-analysis** and **game-test-case-design** skills. QA reviews every output before it reaches Notion.

STEP 1 · TEST ANALYSIS

First, understand the system

I mapped the state model from the live build. It drives both the test design and the automation.



Every box and arrow is a test target. The red boxes are the failure paths (no space, ad fails, game over), which a happy-path pass usually skips.

STEP 1–2 · RISK AND COVERAGE

Coverage goes well beyond the UI

LAYER	RISK	TCS ■ AUTOMATED ■ NOT YET	AUTO / ALL
L1 · Core rules	P0		4/4
L2 · State	P0		2/2
L3 · Scoring	P0		3/3
L4 · UI	P1		3/6
L5 · Input	P0		2/2
L6 · Persistence	P0		2/5
L7 · Ads	P0		2/7
L8 · Analytics	P1		1/3
L9 · Platform	P1		1/4
L10 · Performance	P1		3/4
L11 · Compatibility	P1		0/3
L12 · L10n & a11y	P2		0/1
L13 · Experience	P1		0/3

14 · 27 · 6

P0 · P1 · P2 test cases

23

automated now (bridge, scene or image)

21

candidates: they need an ads sandbox, network capture or a device lab

3

exploratory charters: fairness, drag feel, ad interruption

NO SPEC, SO I WROTE ONE DOWN

Observed rules become the test oracle

R-01	Piece fits only on empty cells inside the 9×9 board	verified
R-02	Full rows, columns and 3×3 boxes clear at the same time	verified
R-03	New hand only after all 3 pieces are used	verified
R-04	+1 point per placed cell	verified
R-05	Clear bonus = 18 × groups + 10 × streak	needs GD sign-off
R-06	No fit → Second Chance → Game Over	verified
R-08	A hand never holds two identical pieces	verified
R-09	Returning player: dashboard → resume the saved board	verified

Open questions for GD / Dev

- **Q-01** Rescue limit, and what happens on reload during the countdown?
- **Q-02** Is ~6 s of "LOADING AD…" before Game Over the intended UX?
- **Q-03** Confirm the scoring formula (R-05)
- **Q-04** Does the dealer guarantee at least one playable piece?
- **Q-05** Can Dev ship a Game Test Bridge in dev/staging builds?

Why a model? The test compares the board with an independent model after *every* move, not with a screenshot. If a rule changes, we see the mismatch instead of a silently green script.

STEP 3 · AUTOMATION

The three-tier ladder from the proposal, on a canvas game

Start from the most stable control point. Step down a tier only when the higher one can't reach.

TIER 1 · GAME TEST BRIDGE

Read and arrange game state

Board as FEN, score, hand, open screens. Sets an exact board + hand instead of relying on random deals.

```
# today: QA shim over Phaser
# in-house: dev-owned contract
game.qa("fen")
→ "x/x/x/x/bbbb1bbbb/x/x/x/x l
s1,b"
game.arrange(board, hand)
```

TIER 2 · SCENE TREE (POCO-STYLE)

Real input, by node

Phaser display list = UI tree. Named nodes, text and cell geometry. Real pointer drags with closed-loop correction.

```
game.tap_node("PlaySingle")
game.qa("screenTexts",
"GAME_OVER_SCREEN")
→ ["48", "48"]
game.place(slot=0, row=4, col=0)
```

TIER 3 · IMAGE (AIRTEST TECHNIQUE)

When nothing else reaches

OpenCV template matching, multi-scale, DPR-aware. Finds the PLAY button, screen titles and checks that the canvas isn't blank.

```
vision.wait_for(page,
"title_your_score")
vision.tap(page, "btn_play")
→ Match(score=0.97)
```

Every tier sits in the same Pytest suite with the same evidence and report. On native Android/iOS builds, tiers 2–3 become Poco SDK / Airtest; the tests and the oracle stay the same.

THE RULE EVERY TEST FOLLOWS

Arrange through the bridge. Act through real input. Assert against the model.

Arrange

Bridge sets the exact board and hand. No random deals, no 5-minute play-throughs to reach a state.

Act

Real pointer drag on the canvas. The driver reads where the piece is during the drag and corrects the pointer, so there are no hard-coded pixels.

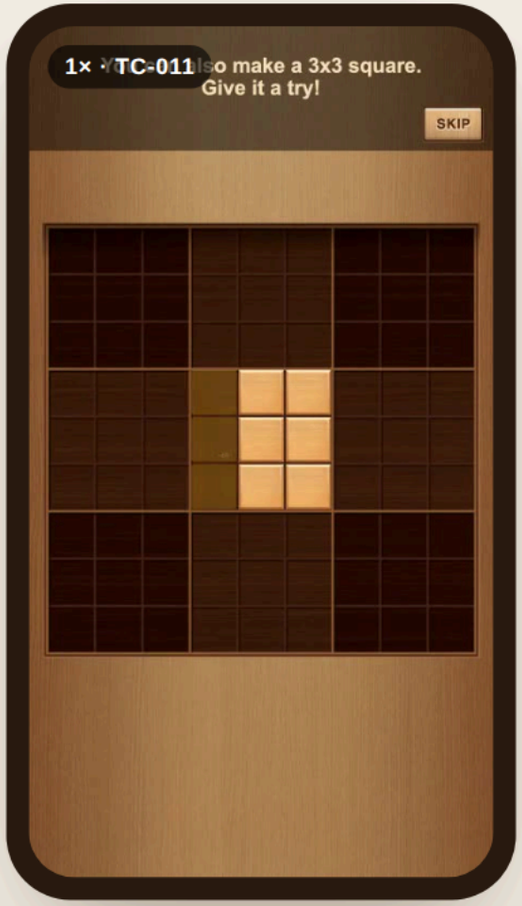
Assert

Board == model after the move, score delta == the formula, state transitions read from the game itself.

```
# tests/test_03_board_rules.py — one real test, unedited logic
def test_line_and_box_clears(ready, kind="row+column"):
    b = rules.empty();  fill row 4 and column 0, leave (4,0) open
    before = ready.arrange(b, hand("square_1"))          # Arrange · bridge
    expected, cleared = rules.place(b, piece, 4, 0)      # oracle · model
    result = ready.place(0, 4, 0)                       # Act · real drag
    assert result["after"]["board"] == expected             # Assert · model
    assert groups_cleared(cleared) == 2
```

RECORDED RUN · REAL SPEED

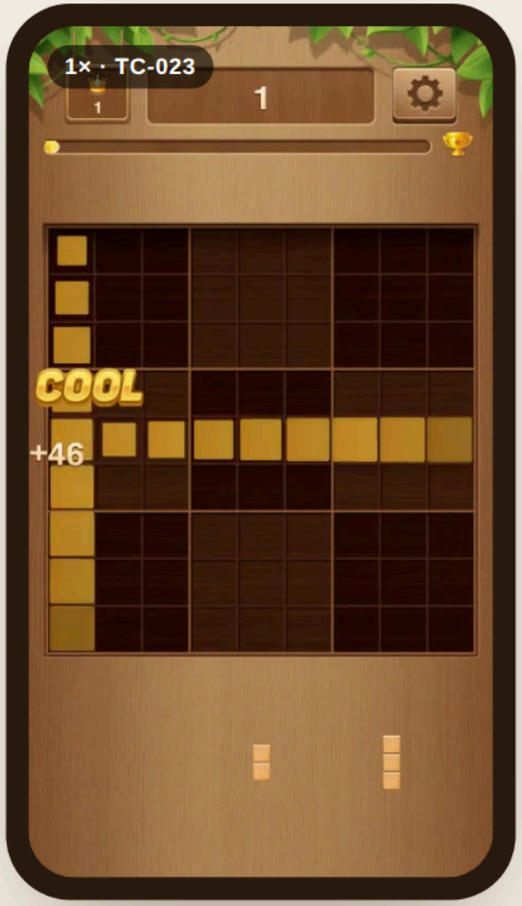
Tutorial with real drags, then a row + column clear



First-time player: 4 tutorial steps

Each step is solved by the model and done with a real drag. The tutorial must award no points and must leave an empty board.

✓ 4 steps · score 0 · 3 pieces dealt



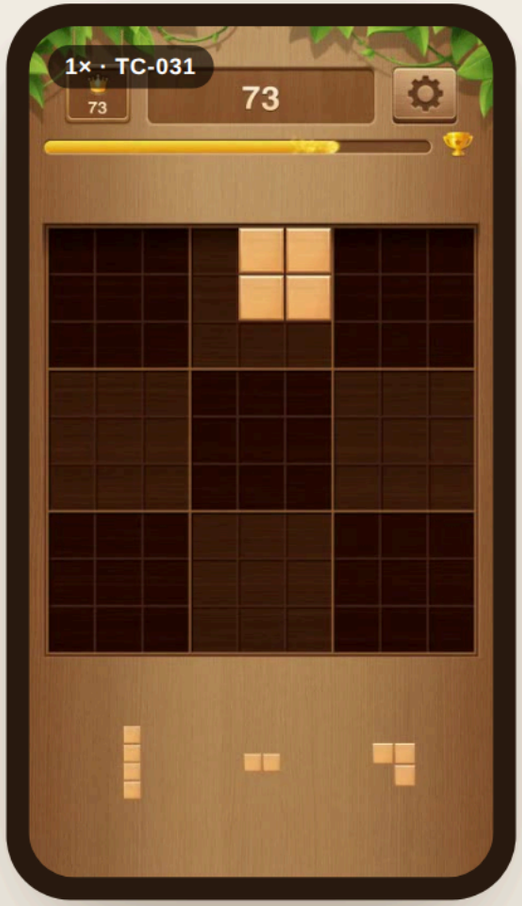
Arranged board, one move

The bridge sets row 4 and column 0 one cell short. One drag should clear both at once.

✓ board == model · 2 groups · +47

RECORDED RUN

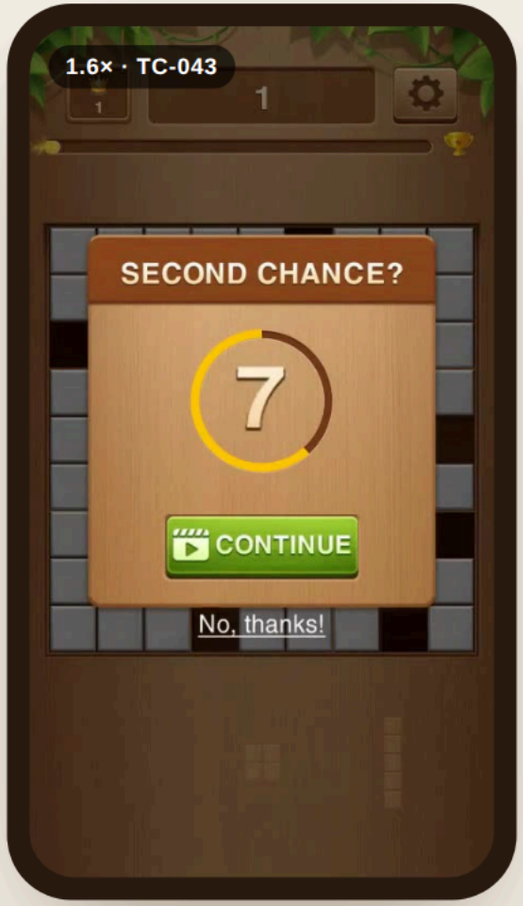
Scoring streak, then Game Over → PLAY by image



Clear, clear, no clear

The score delta is checked against the observed formula on every move: +29, +40, then +4 when the streak resets.

✓ formula holds · flagged for GD (Q-03)



No fit → Second Chance → PLAY

Ads are blocked, so the rewarded ad fails. The player must still reach Game Over. PLAY is then found by OpenCV only, with no tree and no ids.

✓ not blocked · new game: score 0, empty board

STEP 4 · REPORT

Results on the live build

32 / 32
tests passed

9m 42s
full suite · a new player per test

~9.0 s
median Play → playable (8.1–10.8 s)

0
drag retries across 32 tests

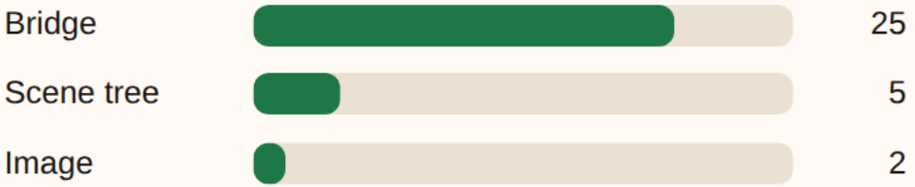
60 fps
avg over 12 moves (GPU Chromium)

+6 MB
JS heap growth over 12 moves

1.95 MB
game payload · 38 resources

23 / 47
test cases covered by this run

Tests by tier



Evidence for every test

Final screenshot, game state (FEN, score, open screens), console errors, blocked ad/analytics requests, video. JUnit feeds the Notion **Automation Run** by TC ID.

HONEST LOG

What the exercise taught us

✓ A false positive I avoided

The Game Over panel showed "0" at first. It counts up, so it isn't a bug. I checked with real gameplay, and the test now asserts after the animation settles.

! Test hooks must respect game rules

Injecting two identical pieces made them share one sprite. The real dealer never does that, so the bridge now enforces the same rule (R-08).

◎ The model caught my own test bug

A 3×3 piece at (0,0) completes box 0: +37, not +9. The oracle flagged it before I trusted the test.

⊘ Block ads and analytics in functional runs

Random ads make runs flaky, and test traffic inflates DAU and ad KPIs. Ads get their own suite with test placements.

⚙ Use GPU runners for anything perf-related

The default headless browser uses software GL: ~40 fps and 7 CPU cores. With a GPU it is 60 fps and game-state calls are ~10× faster.

0 No product defect confirmed in this scope

That is a result too. The value is a repeatable net, plus 5 questions that make the spec explicit.

FROM DEMO TO TEAM PRACTICE

What this means for Sun.Studio

The one ask to Dev: a Game Test Bridge contract

My shim reads the internals of a minified build. That is fine for a demo, but it is not a contract. Expose this in **dev/staging builds only**:

```
getState()        // board, hand, score, screen
setState(fen)     // arrange any situation
seed(n)           // deterministic deals
forceAd(result)   // ok · fail · closed early
```

Why it pays off for Dev too

- A bug report carries a FEN, so Dev reproduces the exact board with one call
- RNG-dependent issues (unfair deals, game-over edge cases) become repeatable
- The same hooks serve balancing tools, the device lab and the CI smoke gate

Reuse per new game

- **Per game:** bridge + rules model, ~1–2 days for a Phaser title
- **Shared:** driver, evidence, report, Notion sync, CI
- Unity / Cocos: Poco SDK + Airtest; tests stay the same

Where it fits in the 90-day plan

- Weeks 3–4: automation spike on one P0 path ← **this is it**
- Two pilot games: P0 suite as the build gate (G2)
- Results flow into the Notion dashboard

THANK YOU

Questions?

Analysis, 47 test cases, 32 tests, report and Notion sync are all in one repository.

`github.com/thanhmimi/wbj-qa-demo`

README · DEMO.md · testcases/ · tests/ · wbj/ · tools/

Thanh Nguyen (Leon) · Quality Engineering Leader · hands-on automation · AI-driven quality

